VR/AR向けエンドツーエンド暗号化の低遅延性評価：MLSと低遅延の暗号学的置換Areionを用いた実機測定

手老 陽南[1,a)]　倉﨑翔大[1]　金岡 晃[1]　酒見 由美[2]　菅野 哲[2]　五十部 孝典[3]

概要：VR/AR 技術の利用は医療や軍事訓練、教育など機密性の高い情報を扱う分野へ拡大している。従来環境ではエンドツーエンド暗号化（E2EE）が広く普及する一方、VR/AR 環境では E2EE の導入に関する議論は十分に行われていない。加えて、その適用に伴う暗号処理が通信遅延の増加を招く懸念もある。VR 環境における遅延の増加は VR 酔いや作業ミスにつながるため遅延への影響を定量的に把握することが導入の前提となる。そこで本稿では、まず VR/AR 環境で E2EE を実現するために必要な要件を整理し、そのうち低遅延性と動的なメンバの参加・脱退への対応の 2 点に着目する。そのうえで、動的な鍵更新を特徴としグループ通信向け E2EE プロトコルである MLS を使用し、E2EE 導入時の遅延を実測評価する。さらに、低遅延暗号 Areion と従来の AES の処理性能を比較し、VR デバイス間通信において低遅延性を担保しうる暗号手法を検討する。

Latency Evaluation of End-to-End Encryption for VR/AR : A Measurement Study with MLS and the Low-Latency Permutation Areion

Hina Terou[1,a)]　Shodai Kurasaki[1]　Akira Kanaoka[1]　Yumi Sakemi[2]　Satoru Kanno[2]　Takanori Isobe[3]

Abstract: The use of VR/AR technology is expanding into fields that handle highly confidential information, such as healthcare, military training, and education. While end-to-end encryption (E2EE) has become widespread in conventional environments, its adoption in VR/AR environments has not been sufficiently discussed. In addition, the cryptographic processing required for E2EE may increase communication latency. Since increased latency in VR environments can lead to VR sickness and operational errors, quantitatively understanding its impact on latency is a prerequisite for adoption. In this paper, we first identify the design requirements for E2EE in VR/AR environments, and focus on two of them: low latency and support for the dynamic joining and leaving of members. We then use Messaging Layer Security (MLS), a group-communication E2EE protocol characterized by dynamic key updates, to experimentally evaluate the latency overhead of E2EE. Furthermore, we compare the performance of the low-latency permutation Areion with that of conventional AES, and examine the suitability of low-latency cryptography for E2EE in VR communications.

1. はじめに

VR/AR 環境では、ユーザ間でテキスト、音声、動画を送受信するだけでなく、オブジェクトの形状や外観、位置および回転、動作を規定するスクリプト、空間内でのユーザの行動といった、3 次元空間を成立させるための情報が共有される。これらの情報は、デジタル空間の構造、ユーザの行動、業務内容等を推測する手掛かりになり得るため、従来のメディアデータと同様に保護する必要がある。

従来のコンピューティング環境では、通信経路上の第三者だけでなく、通信を中継するサービス事業者からも情報を秘匿する手段として、エンドツーエンド暗号化（E2EE）が普及している。一方、VR/AR 環境では、オブジェクト情報や空間同期情報についても、機密性および完全性を確保する必要がある。本研究では、このような 3 次元空間を構

[1] 東邦大学
Toho University
[2] GMO コネクト株式会社 / GMO インターネットグループ株式会社
GMO CONNECT Inc. / GMO Internet Group, Inc.
[3] 大阪大学
The University of Osaka
[a)] 6525008t@st.toho-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

成するデータを含めた E2EE を空間暗号化と呼ぶ。

空間暗号化では、暗号化対象となるデータが多様であることに加え、マルチユーザ VR/AR アプリケーションにおける頻繁なユーザの参加・脱退への対応が必要となる。参加前や脱退後の通信内容へのアクセスを防ぐには、参加者の変更に応じた暗号鍵の更新が必要である。そのため、VR/AR 環境における E2EE では暗号化性能だけでなく動的なメンバ変更、鍵更新、通信中の暗号状態の同期を含むプロトコル全体の実行可能性を検討する必要がある。

しかし、E2EE に伴う暗号処理や鍵更新は、通信遅延を増加させる可能性がある。VR/AR アプリケーションでは、身体動作やオブジェクトの状態変化、映像・音声の提示が連続するため、操作と提示結果の時間的なずれは、操作性や没入感の低下、作業ミス、VR 酔いにつながり得る。そのため、従来のメッセージング環境より遅延への要求が厳しく、E2EE の導入可能性を判断するには、実際の VR デバイス上で暗号処理およびプロトコル処理に要する時間を定量的に評価する必要がある。

さらに、VR/AR アプリケーションにおける暗号処理時間は、暗号アルゴリズム単体の性能だけでは決まらない。一般的な開発環境である Unity では、C#コードから P/Invoke および IL2CPP を介してネイティブライブラリを呼び出すため、関数の呼び出し、データ形式変換やバッファへの格納、メモリ確保、スレッドスケジューリング、描画処理との競合などのオーバーヘッドが生じる。そのため、ネイティブ実行による暗号プリミティブの評価だけでは、実際の VR アプリケーションにおける処理時間を推定することは困難である。E2EE の導入可能性を検討するには、暗号処理の直接実行に加え、実際の VR/AR アプリケーションから暗号ライブラリを呼び出す場合も評価する必要がある。

暗号実装の性能評価では、スループットや 1byte 当たりの CPU サイクル数が主な指標として用いられてきた。しかし、これらは総合的な処理能力を示す指標であり、データ入力から最初の暗号文が出力されるまでの待ち時間を直接評価するものではない。そのため、高スループットな実装でも、初期化処理や内部バッファリング等により、最初の出力が早いとは限らない。

この違いは、空間同期情報と連続的なメディアデータが混在する VR/AR 環境において重要となる。オブジェクトの位置、状態、音声、映像などのデータが繰り返し暗号化されるため、全体の処理能力だけでなく、暗号処理の開始から最初の暗号文が得られるまでの時間も、アプリケーションの応答性を評価する重要な指標となる。

以上を踏まえ本研究では、以下の 2 つのリサーチクエスチョン（RQ）を設定する。

- RQ1：VR 環境に E2EE を導入した場合、動的なメンバ変更および鍵更新を含むプロトコル処理にどの程度の時間を要し、その処理時間は実装形態によってどのように変化するか。
- RQ2：VR 環境においても低遅延性を担保しうる暗号手法は何か。

RQ1 に対しては、動的なメンバ変更と鍵更新を特徴とするグループ通信向け E2EE プロトコルの MLS（Messaging Layer Security）を使用する。2 台の Meta Quest 3 を同一の無線 LAN へ接続し、メンバの参加、脱退、および鍵更新に要する処理時間を実測する。また、Android 向けネイティブ実行ファイルから MLS 処理を直接実行する構成と、Unity 上の C#コードから P/Invoke および IL2CPP を介して実行する構成を比較する。

RQ2 に対しては、低遅延な暗号学的置換 Areion を Meta Quest 3 上で動作させ、AEAD 方式である Areion-256-OPP および Areion-512-OPP の性能を評価する。比較対象には AES-128-GCM および AES-256-GCM を用いる。さらに、暗号処理を直接実行した場合と、P/Invoke および IL2CPP を介して実行した場合の処理時間を比較する。

その結果、RQ1 に対しては、VR 環境での E2EE の処理時間が MLS のプロトコル処理そのものだけでなく、IL2CPP を介したネイティブライブラリの呼び出し方法や処理の実行タイミングに大きく依存することが示された。

RQ2 に対しては、Areion が最初の暗号文出力までの時間において AES-GCM を上回る一方、全体のスループットでは下回った。ただし、Areion の公開実装は AArch64 上で動作するものの、AArch64 専用の高度な最適化実装ではなく、reference C 実装をコンパイラで最適化した構成である。一方、BoringSSL の AES-GCM は ARM Cryptography Extensions を利用する AArch64 向け最適化実装である。そのため、両者の性能差には暗号方式だけでなく、実装成熟度およびアーキテクチャ固有の最適化水準の差が含まれることがわかった。

以上の結果から、ネイティブライブラリの適切な利用や鍵更新時の暗号状態の同期を考慮することで、E2EE に伴う処理がユーザ体験へ与える影響を抑えられる可能性があることが示された。

2. 関連研究

2.1 VR/AR 環境におけるセキュリティとプライバシー

VR/AR 技術は、その発展に伴ってセキュリティ対策やプライバシーに関する研究 [1] が進んでいるが、未だその重要性は広く認識されていない。金岡らは VR/AR 環境におけるセキュリティリスクを体系的に分類し、VR/AR 環境でリスクが高まると考えられる脅威を列挙している [2]。

© 2026 Information Processing Society of Japan

2.2 従来環境における E2EE の普及と VR/AR 環境における E2EE の重要性

従来環境で利用されるメッセージングアプリの WhatsApp と Signal では、デフォルトで全てのメッセージングと通話に E2EE を使用している。また、E2EE はメッセージングアプリだけではなく Web 会議アプリケーションの Zoom や Jitsi Meet でも利用可能になっている [3]。例えば Zoom では通常、全ての共有コンテンツがユーザとサーバ間で暗号化されている。しかし、一部の機能制限を伴うオプション機能として E2EE が提供されており、高い機密性が求められる場合にはユーザが有効化することが可能である [4]。

音声、ビデオ、テキストを用いたコミュニケーションツールである Discord では 2024 年 9 月より E2EE プロトコル DAVE（Discord Audio and Video End-to-End Encryption）[5] の運用を開始した。DAVE はグループ鍵交換に MLS を採用し、メディアフレームの暗号化には WebRTC の Encoded Transform API を用いる。MLS の暗号スイートは `DHKEMP256_AES128GCM_SHA256_P256` であり、メディアフレームは AES-128-GCM によって暗号化される。さらに Discord は Meta Quest 向けのネイティブアプリを公開しており、ボイスチャットを含む機能が VR デバイス上で利用可能となっている [6]。DAVE の仕様書はどのプラットフォームにおいても本プロトコルへ対応することを前提としているため、Meta Quest 上でも DAVE の適用対象になると考えられる。MLS を用いた E2EE は VR デバイス上ですでに動作している一方、その処理時間や遅延に関する情報は公表されていない。

このように、テキストデータだけでなく音声や動画データの通信でも E2EE の普及が進んでおり、その重要性は今後さらに高まると考えられる。

このように、音声や動画を含む通信でも E2EE の利用が進んでいる。VR/AR 環境におけるマルチユーザ通信ではクラウドを介したデータ交換が一般的であるため、クラウドを中継した場合でも VR/AR デバイス間でデータを保護できる E2EE が必要となる。

2.3 通信遅延が VR/AR 体験に与える影響

VR/AR 環境では、通信遅延の増加が操作性やユーザ体験に影響を及ぼす。Sajid らは、MR 環境に約 10ms の可変レイテンシと 0.5%のパケット損失を導入することで、参加者に強い不満が生じることを示している [7]。また、Motion-to-photon latency については 20msec 以内が推奨されている [8, 9]。遠隔操作では、通信遅延が 100ms を超えると操作性への影響が現れ、300ms を超えると作業時間やエラー率への影響が顕著になることが報告されている [10]。遠隔手術ガイドラインでも、通信遅延 100ms 以下が許容可能な水準として示されている [11]。

2.4 低遅延暗号技術の現状

遅延による影響を小さくする方法の一つとして、低遅延暗号の使用が挙げられる。低遅延暗号に関する研究は、実装対象であるハードウェアとソフトウェアで設計や評価指標が大きく異なり、それぞれが独立して発展している。

ハードウェア実装においては、主に ASIC で暗号化回路を展開して実装するループアンローリングが主流となっている。例えば、10 ラウンド構成の AES を実装する場合、同一のラウンド関数を反復するのではなく 10 ラウンド分の回路を直列に配置することで、1 サイクルでの暗号化完了を目的としている。この手法における主な評価指標は、展開された回路におけるクリティカルパスの遅延時間となる。代表的な低遅延暗号アルゴリズムとして、PRINCE、QARMA、Dialga などが挙げられる。

一方、ソフトウェア実装における低遅延性とは、主にショートメッセージ暗号化時においてデータが入力されてから最初の暗号文が出力されるまでの処理時間を指す。明示的にソフトウェア環境での低遅延性を示す暗号アルゴリズムとしては Areion [12, 13] が挙げられる。

3. 空間暗号化の必要性

3.1 脅威分析：E2EE を要する VR/AR 環境での脅威

本節では、VR/AR 環境における脅威のうち、E2EE による対策が有効なものを明らかにする。分析にあたっては、VR/AR 環境の脅威を体系的に整理した金岡らの分類 [2] の 49 個の脅威から、E2EE による対策が有効な脅威を抽出した。その結果、「通信データの不正取得」「複数ユーザ環境での情報漏洩」「通信データの改ざん」が E2EE による対策が有効な脅威ということが分かった。VR/AR 環境で利用されている主要な通信方式は TLS であり、これらの脅威は TLS では対処することが不可能である。したがって、VR/AR 環境においても E2EE を導入する必要性が高い。

3.2 E2EE 対象データの定義

VR/AR 空間は多様なデータによって構成される。例えば、空間内に配置されるオブジェクトを表現する 3D オブジェクトデータ、ユーザの生体データ、ユーザの行動履歴データ、マルチユーザ環境における同期状態などが挙げられる。これらはいずれも VR/AR 空間の成立に寄与するデータであるが、そのすべてを E2EE の対象とする必要性はないと考える。

そこで、E2EE の対象とすべきデータの範囲を明確にする。暗号化の対象を、空間共有のために相手へ送信することが必須なオブジェクトデータと考えると、オブジェクトの形状および外観を構成する情報、オブジェクトの位置および回転を表す座標データ、可動オブジェクトにおける速度および質量、オブジェクトの挙動を規定するスクリプト

© 2026 Information Processing Society of Japan

が暗号化対象データとなると考えられる。これらの情報は、通信時に相手の空間において同一のオブジェクトを再現するために送信が必須である。さらに、VR/AR 環境では複数のユーザが同一の空間を共有し、その空間を構成するデータを相互に送信し合うことでコミュニケーションが成立する。したがって、VR/AR 環境における E2EE の対象は、テキスト、音声、動画に加え、空間も対象とする必要がある。

また、ユーザが他ユーザの空間へ気軽に参加できる形態のサービスでは、同時に参加するユーザの参加と脱退が頻繁に発生する。そのため、VR/AR 環境では暗号化対象メディアや参加ユーザの切り替わりに対応する必要がある。本研究では、上記で定義した E2EE 対象データに対して施す E2EE を空間暗号化と呼ぶこととする。

3.3 空間暗号化の設計要件

2.3 節や 3.2 節を踏まえ、空間暗号化を実現するための設計要件を以下の 4 つと置いた。

(1) 低遅延
(2) 参加者の参加と脱退の動的な対応
(3) 暗号化対象メディアの切り替え対応
(4) 一般的な VR プラットフォームにおける実装が可能

本研究では、このうち要件 1 と要件 2 に着目した。

4. 実験デザイン

RQ を明らかにするため、VR 環境での E2EE 導入時の遅延評価と、低遅延な暗号学的置換 Areion の低遅延性の効果を評価する 2 つの実験を行った。なお本研究では、MLS の暗号スイートを Areion へ置き換えた統合評価は行っていない。

4.1 実験 1：MLS のパフォーマンス評価

4.1.1 対象プロトコルと選定理由

本実験では、E2EE を実現するプロトコルとして MLS を使用した。MLS は、二分木構造のラチェットツリーを用いることで、グループのメンバ数に対して対数オーダで鍵更新を可能にするグループ通信向け E2EE プロトコルである。鍵更新は、メンバが鍵更新を提案し、それを管理者がコミットすることによって行われる。この機構により、メンバの参加および脱退が発生した場合にも、参加以前の通信内容を新規参加者が復号できないこと、および脱退以後の通信内容を脱退者が復号できないことが保証される。このように動的なメンバ変化への対応を備えている点や、IETF によって RFC 9420 として標準化されている点から、本研究の実験では E2EE プロトコルとして MLS を採用した。

4.1.2 実装方法

本実験における MLS の実装には、Rust で記述されたオープンソース実装である OpenMLS を用いた。ソースコードを Android ネイティブライブラリとしてビルドし、Android ネイティブライブラリから関数を呼び出すことで MLS の各処理を実行する。ここで、MLS の呼び出し方の違いが処理時間に与える影響を評価するため 2 通りの構成で実装した。1 つ目の構成では、Android ネイティブライブラリ化した OpenMLS を、Unity 上の C#スクリプトから呼び出す。これは Unity を用いた VR アプリケーション開発において一般的な実装形態であり、実運用に近い条件での処理時間が評価可能である。2 つ目の構成では、Android 向けにクロスコンパイルした実行ファイルから、Android ネイティブライブラリを直接実行する。この構成では Unity ランタイムを経由しないため、デバイスの処理性能が評価可能である。実装方法の詳細は付録 A.1 に示す。

4.1.3 評価環境

実験を行うデバイスには Meta Quest 3 を 2 台使用した。1 台をグループの作成やコミットを行う管理者とグループメンバ、もう 1 台をグループへの参加および脱退を行うメンバとして割り当てる。2 台は同一の IEEE 802.11ax の無線 LAN アクセスポイントに接続し、このアクセスポイントには実験対象以外のデバイスを接続していない。デバイス間の通信は、名前解決による影響を排除するため IP アドレスを直接指定して行う。

4.1.4 評価項目

計測対象とする処理は、メンバの参加処理、メンバの脱退処理、および鍵更新処理である。鍵更新については、メンバが鍵更新提案を行い管理者がコミットを行う場合と、管理者が鍵更新提案とコミットを行う場合の 2 パターンについて計測する。また、グループ人数は 2 人、3 人、5 人でそれぞれの処理を計測する。計測する項目は各処理に要した時間である。計測は各条件につき 1010 回実施し、このうち 1 回目から 10 回目まではウォームアップとして評価対象から除外し、残る 1000 回分を評価対象とする。

4.2 実験 2：Areion のパフォーマンス評価

4.2.1 対象方式と実装方法

本実験では、Areion の Meta Quest 3 上での基礎性能を確認し、AEAD として利用した場合の処理性能を AES-GCM と比較する。Areion には GitHub の実装の ref ターゲットを用いた[*1]。Areion-256 および Areion-512 の置換、DM 圧縮関数、Areion MD に加え、AEAD 方式である Areion-256-OPP および Areion-512-OPP を評価対象とした。ref ターゲットは AArch64 環境において NEON intrinsics および ARM Cryptography Extensions の AES 命令を使用する。一方、手書きの AArch64 アセンブリや複数ブロック処理に特化した実装は使用していない。

比較対象には AES-128-GCM および AES-256-GCM を用

[*1] https://github.com/gmo-ierae/low-latency-crypto-areion

© 2026 Information Processing Society of Japan

いた。直接実行のスループット評価では、Android 上の BoringSSL が提供する AES-GCM 実装を使用した。

さらに、一般的な Unity アプリケーションから暗号処理を利用する場合の処理時間を確認するため、Areion-256-OPP と AES-128-GCM を含む Android ネイティブライブラリを作成し、Unity 上の C#スクリプトから P/Invoke で呼び出した。実装方法の詳細は付録 A.2 に示す。

4.2.2 評価環境と評価項目

Meta Quest 3 においてアプリケーションから利用可能な暗号関連命令を確認するため、C/C++で調査プログラムを作成した。Linux の `getauxval()` により `AT_HWCAP` および `AT_HWCAP2` を取得するとともに、Google の `cpu_features` ライブラリによる判定結果と照合した。

Areion の基礎性能では、1byte 当たりの CPU サイクル数を示す cycles/byte を計測した。各測定実行では同一処理を 20,000 回反復し、合計サイクル数を合計処理 byte 数で除して 1 つの cycles/byte を算出した。

AEAD のスループット評価では、payload を 1、10、100、200、300、400、500KB とし、AAD を 16byte、認証 tag を 16byte とした。AES-GCM の nonce は 12byte、Areion-OPP の nonce は 16byte である。各条件で 500ms のウォームアップを行った後、各暗号化対象データサイズに対して、1KB および 10KB では 20,000 回反復した。100KB 以上では 1 回の測定実行における合計処理量が約 256MB となるよう反復回数を設定した。各条件を別プロセスで 5 回測定し、スループットの中央値を採用した。

評価指標には、暗号化開始から最初の暗号文が出力されるまでの時間を用いた。これを Time to First Ciphertext（TTFC）と呼ぶこととする。最初の入力を 32byte、AAD を 0byte とし、初期化済み state または context に対する最初の update 処理のみを計測する条件と、鍵および nonce の設定開始から最初の暗号文出力までを計測する条件を設けた。各条件を 10,000 サンプル計測し、中央値を採用した。

Unity 経由実行では、暗号処理を直接実行した場合と、C#から P/Invoke および IL2CPP を経由して実行した場合を比較した。処理時間の計測には、IL2CPP 経由実行では C#の System.Diagnostics.Stopwatch.GetTimestamp() を用い、取得した値を Stopwatch.Frequency によりナノ秒へ換算した。換算の単位が 100ns となるため、IL2CPP 経由実行について得られる処理時間はすべて 100ns 単位に量子化された値となる。各処理は context の生成、初期化、AAD 処理、暗号化または復号、tag 生成または検証および context 解放を含む one-shot AEAD 処理である。

5. 実験結果

5.1 実験 1:MLS の評価

管理者を admin、参加や脱退および鍵更新提案処理を行うメンバを member、グループ人数が 3 人と 5 人の場合のメンバを guest とする。また、各結果において複数の guest 間で処理時間に顕著な差が認められなかったため、代表して 1 名分の結果のみを示す。なお、以降の各表に示す処理時間の値はいずれもミリ秒（ms）である。

表 1: IL2CPP 経由実行したときの参加処理時間の結果

役割	人数	平均	中央	最大	P95	P99
admin	2	13.90	13.91	14.79	14.30	14.52
admin	3	26.74	26.61	29.12	27.86	28.39
admin	5	56.99	56.82	60.95	58.33	58.98
member	2	85.65	70.53	153.21	126.25	139.50
member	3	80.77	70.48	153.29	126.19	126.68
member	5	98.62	112.07	223.48	126.46	140.09
guest0	3	1.17	1.15	2.73	1.27	1.73
guest0	5	1.21	1.20	2.80	1.32	1.51

表 2: 直接実行したときの参加処理時間の結果

役割	人数	平均	中央	最大	P95	P99
admin	2	1.32	1.17	2.80	1.94	2.25
admin	3	1.77	1.72	2.88	2.26	2.54
admin	5	2.01	1.93	3.34	2.60	3.03
member	2	11.76	8.93	152.42	14.73	140.81
member	3	12.79	9.36	154.42	14.30	146.65
member	5	12.89	9.54	157.06	17.11	148.01
guest0	3	1.12	1.04	2.53	1.60	1.90
guest0	5	1.32	1.14	5.15	2.36	3.31

表 3: IL2CPP 経由実行したときのメンバからの提案による鍵更新にかかる処理時間

役割	人数	平均	中央	最大	P95	P99
admin	2	13.85	13.85	15.64	14.39	14.94
admin	3	40.86	40.79	42.95	41.92	42.22
admin	5	97.75	97.62	100.25	98.93	99.46
member	2	107.13	124.52	222.31	138.93	166.09
member	3	130.57	125.02	471.99	140.00	235.63
member	5	207.22	208.20	277.24	235.78	249.03
guest0	3	0.97	0.94	2.45	1.05	1.88
guest0	5	1.00	0.99	1.98	1.10	1.18

5.1.1 メンバ参加処理に関する結果

IL2CPP 経由実行したときの処理時間の結果を表 1 に、直接実行したときの処理時間の結果を表 2 に記載する。

5.1.2 メンバからの提案による鍵更新に関する結果

IL2CPP 経由実行したときの処理時間の結果を表 3 に、直接実行したときの処理時間の結果を表 4 に記載する。

5.1.3 管理者からの提案による鍵更新に関する結果

IL2CPP 経由実行したときの処理時間の結果を表 5 に、直接実行したときの処理時間の結果を表 6 に記載する。また、管理者からの提案による鍵更新の 3 人構成では、guest の処理時間取得時にプロトコルエラーにより完了しなかっ

© 2026 Information Processing Society of Japan

表 4: 直接実行したときのメンバからの提案による鍵更新にかかる処理時間

役割	人数	平均	中央	最大	P95	P99
admin	2	1.23	1.11	2.36	1.77	2.00
admin	3	1.66	1.62	3.09	2.14	2.50
admin	5	1.98	1.88	4.61	2.86	3.22
member	2	11.12	8.38	152.24	13.54	136.67
member	3	12.96	9.16	155.26	18.53	141.81
member	5	12.61	9.70	152.74	15.98	140.80
guest0	3	0.87	0.81	2.52	1.23	1.46
guest0	5	1.16	0.97	3.57	2.12	2.85

表 5: IL2CPP 経由実行したとき管理者からの提案による鍵更新にかかる処理時間

役割	人数	平均	中央	最大	P95	P99
admin	2	12.30	12.23	14.75	13.15	14.08
admin	3	26.89	26.88	29.24	28.29	28.73
admin	5	56.56	56.63	58.68	57.84	58.23
member	2	0.76	0.75	1.82	0.81	0.90
member	3	0.78	0.77	1.03	0.83	0.91
member	5	0.84	0.83	1.08	0.90	0.96
guest0	3	0.91	0.89	2.45	0.98	1.89
guest0	5	1.00	0.97	1.43	1.21	1.34

表 6: 直接実行したとき管理者からの提案による鍵更新にかかる処理時間

役割	人数	平均	中央	最大	P95	P99
admin	2	0.76	0.69	1.54	1.15	1.39
admin	3	1.69	1.66	4.22	2.45	2.90
admin	5	3.42	3.33	8.95	5.01	5.78
member	2	0.80	0.75	1.69	1.15	1.36
member	3	0.85	0.78	1.67	1.22	1.43
member	5	0.91	0.86	1.84	1.27	1.44
guest0	3	0.89	0.83	1.77	1.26	1.47
guest0	5	1.02	0.92	3.60	1.62	2.45

た実行が IL2CPP 経由実行したときで 9 回、直接実行したときに 2 回含まれていた。これらは評価対象から除外した。

5.1.4 メンバの脱退処理に関する結果

IL2CPP 経由実行したときの処理時間の結果を表 7 に、直接実行したときの処理時間の結果を表 8 に記載する。

5.1.5 実験 1 の結果のまとめ

計測結果から処理時間は、グループ人数の増加に伴って概ね増加していることがわかる。実行環境による差は役割によって異なり、admin および提案を行う member では IL2CPP 経由実行時の中央値がいずれの処理においても直接実行時の結果を大きく上回っている。一方 guest では、すべての処理において両環境の中央値の差が 0.2 ms 以内であり、実行環境による大きな差は認められなかった。

表 7: IL2CPP 経由実行したときのメンバの脱退にかかる処理時間

役割	人数	平均	中央	最大	P95	P99
admin	2	13.81	13.82	16.07	15.28	15.59
admin	3	41.03	40.98	43.90	42.68	43.15
admin	5	97.59	97.53	101.97	100.39	101.46
member	2	106.06	111.36	277.33	138.73	180.06
member	3	138.54	138.14	277.65	179.15	249.75
member	5	205.96	207.88	305.17	222.98	249.47
guest0	3	0.75	0.73	1.78	0.82	1.31
guest0	5	0.91	0.90	1.88	1.00	1.08

表 8: 直接実行したときのメンバの脱退にかかる処理時間

役割	人数	平均	中央	最大	P95	P99
admin	2	0.82	0.75	1.77	1.16	1.38
admin	3	1.23	1.12	2.22	1.77	1.99
admin	5	1.67	1.60	4.46	2.26	2.77
member	2	10.60	7.67	153.91	12.07	136.32
member	3	10.81	8.04	156.01	12.21	142.42
member	5	12.47	9.17	155.81	18.32	143.37
guest0	3	0.73	0.65	1.88	1.12	1.41
guest0	5	1.04	0.88	4.36	1.88	2.70

表 9: Meta Quest 3 における Areion の基礎性能（cycles/byte）

処理	性能
Areion-256 permutation	2.98
Areion-512 permutation	1.98
Areion-256 DM	3.02
Areion-512 DM	1.85
Areion MD（32byte）	7.88
Areion MD（4096byte）	3.92

5.2 実験 2：Areion の評価

5.2.1 暗号関連ハードウェア支援の調査

`getauxval()` と `cpu_features` の両方で、AES、PMULL、SHA1、SHA2/SHA256 および CRC32 が利用可能と判定された。このことから、Meta Quest 3 で ARMv8 の暗号関連命令を利用できることを確認した。

5.2.2 Areion の基礎性能

Areion の基礎性能評価結果を表 9 に示す。なお、permutation の評価では並列化は行わず逐次処理により計測した。

5.2.3 AEAD 暗号化スループット

Meta Quest 3 上で直接実行した AEAD 暗号化スループットを表 10 に示す。表 10 に示すスループットは CPU サイクル数（cycles/byte）から Mbps へ換算した値である。 換算時は CPU の周波数を Meta Quest 3 の SoC の最大周波数である 2.36GHz とした。10KB 以上では各方式のスループットは概ね一定となった。100KB から 500KB では、AES-128-GCM が約 19.7Gbps、AES-256-GCM が約 17.5Gbps、Areion-256-

© 2026 Information Processing Society of Japan

表 10: Meta Quest 3 上で直接実行した AEAD 暗号化スループット（Mbps）

データサイズ	AES-128 -GCM	AES-256 -GCM	Areion-256 -OPP	Areion-512 -OPP
1KB	16,450	14,803	8,524	8,983
10KB	19,431	17,175	9,574	11,048
100KB	19,695	17,475	9,705	11,302
200KB	19,736	17,476	9,714	11,279
300KB	19,742	17,517	9,722	11,302
400KB	19,758	17,489	9,708	11,312
500KB	19,677	17,479	9,718	11,255

表 11: 最初の 32byte 入力に対する TTFC（ns）

方式	nonce	初期化なし	初期化あり
Areion-256-OPP	16byte	39	81
AES-128-GCM	12byte	52	240
AES-128-GCM	16byte	52	279

OPP が約 9.7Gbps、Areion-512-OPP が約 11.3Gbps であった。すべてのデータサイズで AES-GCM が Areion-OPP を上回った結果だが、Areion は AArch64（ARM）向けコードはあるが AArch64 専用の最適化実装はされていない点に留意が必要である。

5.2.4 最初の暗号文出力までの遅延

最初の 32byte を入力したときの TTFC を表 11 に示す。初期化を含まない場合、Areion-256-OPP の中央値は 39ns、AES-128-GCM は nonce 長によらず 52ns であった。初期化を含む場合、Areion-256-OPP は 81ns、12byte nonce の AES-128-GCM は 240ns、16byte nonce では 279ns であった。この結果から、最初の暗号文出力については Areion-256-OPP が AES-128-GCM より短いことが確認された。

5.2.5 直接実行と IL2CPP 経由実行の比較

Unity アプリケーション上で C#からネイティブライブラリを呼び出した場合の暗号化処理時間を表 12 に示す。データサイズ 32byte、1KB、64KB のすべての条件で、IL2CPP 経由実行の処理時間は直接実行より大きい値となった。

6. 考察

6.1 実験 1 からの考察

前章の実験 1 の結果より、直接実行した場合と IL2CPP 経由実行の場合では処理時間に大きな差があることが示された。直接実行時は admin および guest の中央値はすべての条件で 3.33ms 以下であった。member においても admin および guest より処理時間は増加しているものの、中央値はすべての条件で 9.70ms 以下であった。この結果は、直接実行する実装形態では遅延がユーザの VR 体験を損なう可能性はほとんどないことを示唆している。

IL2CPP 経由実行時に処理時間を要していたのは、5 人構成における member の処理であり、メンバからの提案による鍵更新で中央値 208.20ms、P95 が 235.78ms、メンバの脱退時で中央値 207.88ms、P95 が 222.98ms であった。

MLS の参加・脱退処理では 200ms を超える遅延が確認されたが、実際にはユーザ操作や UI・アバターの状態遷移と同時に行われるため、そのまま遅延として知覚されるとは限らない。一方、こうした状態遷移を伴わない鍵更新では遅延が知覚される可能性があり、実行タイミングなどアプリケーション設計上の考慮が必要となる。

6.2 実験 2 からの考察

Areion-256-OPP は、最初の 32byte の暗号文出力では AES-128-GCM より短い TTFC を示した。一方、メッセージ全体の暗号化スループットでは、すべての payload サイズで AES-GCM が Areion-OPP を上回った。この結果は、最初の出力が得られるまでの遅延と、全データの処理性能が異なる性質であることを示している。Areion の低遅延性を評価する場合、TTFC と暗号化完了時間を区別する必要がある。また、Areion-OPP は 32byte 単位で出力するため、32byte 未満の入力では入力の蓄積または finalize 処理まで最初の出力が得られない点にも注意が必要である。

大容量データにおいては AES-GCM の方が高いスループットを示した。Areion の公開実装は NEON および ARM Cryptography Extensions を利用しているが、intrinsics を中心とした実装であり、手書き AArch64 アセンブリや複数ブロック処理に特化した最適化は提供されていない。一方、BoringSSL の AES-GCM は AArch64 向けに高度に最適化された実装である。この実装最適化の成熟度の差が、AES-GCM が Areion-OPP をスループットで上回った一因と考えられる。本結果は同一の最適化水準にある暗号プリミティブ同士の比較ではなく、Meta Quest 3 上で現時点に利用可能な実装を用いた場合の処理性能の比較として位置付ける必要がある。したがって、AES-GCM の方が高速であったという結果のみから、Areion という暗号方式自体の性能が AES-GCM より劣ると断定することはできない。

Unity 経由実行では直接実行より処理時間が増加したものの、中央値の最大値は Areion-256-OPP の約 0.14 ms であり、MLS の参加・脱退や鍵更新に要する処理時間と比較して十分に小さい。このため、Unity 経由の暗号処理によるオーバーヘッドが E2EE 導入時の遅延に与える影響は限定的であると考えられる。

7. まとめ

本研究では、3 次元空間データを含む E2EE を空間暗号化と位置付け、低遅延性と動的なメンバ変更への対応に着目して Meta Quest 3 上で MLS を評価した。MLS 関連処理を直接実行した場合、主要な処理時間の中央値は 9.70ms 以下であったが、IL2CPP 経由実行では鍵更新処理が 208.20ms

© 2026 Information Processing Society of Japan

表 12: 直接実行と Unity IL2CPP 経由実行の暗号化処理時間（ns）

データサイズ	方式	実行経路	平均	中央	最大	P95	P99
32byte	Areion-256-OPP	直接実行	312.3	312	5,364	313	417
		IL2CPP 経由	584.7	500	38,700	800	4,800
	AES-128-GCM	直接実行	702.0	573	60,625	573	1,198
		IL2CPP 経由	1,048.0	900	13,500	1,100	8,900
1KB	Areion-256-OPP	直接実行	1,505.3	1,510	1,771	1,511	1,563
		IL2CPP 経由	2,959.6	2,700	39,300	5,300	8,400
	AES-128-GCM	直接実行	1,435.8	990	374,531	1,042	1,198
		IL2CPP 経由	2,491.2	2,100	42,400	2,400	12,500
64KB	Areion-256-OPP	直接実行	83,348.4	78,230	284,844	119,375	174,219
		IL2CPP 経由	145,096.4	139,400	320,300	174,600	210,300
	AES-128-GCM	直接実行	34,066.9	30,469	412,604	31,719	155,157
		IL2CPP 経由	88,582.6	85,400	172,400	110,500	133,900

となる条件が確認された。この結果から、MLS のプロトコル処理自体よりも Unity の実装形態が遅延に大きく影響することが示された。実験 2 の結果は、空間データのサイズ帯によって適する暗号方式が異なることを示している。表 10 の 100KB 以上の条件では AES-GCM が Areion-OPP を上回った一方、表 12 の 32byte 条件では Areion-256-OPP が 312ns、AES-128-GCM が 573ns と Areion が優位であった。ただし、両者には AArch64 向け最適化水準の差があるため、本結果は Meta Quest 3 上で利用可能な実装同士の比較として位置付けられる。

VR/AR 環境において 2.3 節の Motion-to-photon latency の制約を受けるのは、位置および回転を表す 28byte 程度の姿勢データを 90fps で送信する通信であり、大容量データの転送は主にオブジェクトの生成時に限られる。したがって本稿の結果は、高頻度に送信される小容量の空間データを対象とする場合に低遅延暗号の適用が有効となりうることを示している。ただし、実際にこの条件が成立するかは、暗号化の粒度、鍵の割り当て、鍵更新の頻度といった E2EE を VR に適用する際の安全性設計に依存する。これらの設計と、MLS の暗号スイートへの Areion の統合は今後の課題である。

謝辞　本研究は、JST CREST JPMJCR22M4 の支援を受けたものである。

参考文献

[1] Garrido, G. M., Nair, V. and Song, D.: SoK: Data Privacy in Virtual Reality, *Proceedings on Privacy Enhancing Technologies*, Vol. 2024, No. 1, pp. 21–40 (2024).

[2] 金岡 晃，森 亮太，大塚 航世，倉﨑 翔大，大木 哲史，大東 俊博，磯原 隆将：VR/AR 環境におけるセキュリティ脅威と攻撃対象領域の体系的分析，暗号と情報セキュリティシンポジウム (SCIS) 2025 (2025).

[3] Jitsi: Does Jitsi support end-to-end encryption?, `https://jitsi.org/e2ee-in-jitsi/` (2020). Accessed: 2026-04-09.

[4] Zoom Video Communications, Inc.: End-to-End (E2EE) Encryption for Video Meetings, `https://support.zoom.com/hc/en/article?id=zm_kb&sysparm_article=KB0065408` (2024). Accessed: 2026-04-09.

[5] Discord Inc.: Discord Audio & Video End-to-End Encryption (DAVE) Protocol Whitepaper, `https://daveprotocol.com/` (2024). Protocol version 1.1.4. Also available at `https://github.com/discord/dave-protocol`. Accessed: 2026-08-20.

[6] Discord Inc.: Discord on Meta Quest FAQ, `https://support.discord.com/hc/en-us/articles/41412722792983-Discord-on-Meta-Quest-FAQ` (2026). Discord Help Center. Last updated: 2026-06-27. Accessed: 2026-08-20.

[7] Sajid, M., Bukhari, S. I. M. S., Ji, B. and David-John, B.: "Just stop doing everything for now!": Understanding security attacks in remote collaborative mixed reality, *Proceedings of the 2025 IEEE Conference Virtual Reality and 3D User Interfaces (VR)*, IEEE, pp. 623–633 (2025).

[8] Unity Technologies: Motion-to-photon latency, `https://unity.com/ja/glossary/motion-to-photon-latency` (2026).

[9] Varjo Technologies Oy: Latency in VR/AR, `https://support.varjo.com/hc/en-us/latency` (2026).

[10] Takahashi, Y., Hakamada, K., Morohashi, H., Wakasa, Y., Fujita, H., Ebihara, Y., Oki, E., Hirano, S. and Mori, M.: Effects of communication delay in the dual cockpit remote robotic surgery system, *Surgery Today*, Vol. 54, No. 5, pp. 496–501 (2024).

[11] 日本外科学会遠隔手術実施推進委員会：遠隔手術ガイドライン 改訂第 2 版，診療ガイドライン，一般社団法人日本外科学会，東京 (2025).

[12] Isobe, T., Ito, R., Liu, F., Minematsu, K., Nakahashi, M., Sakamoto, K. and Shiba, R.: Areion: Highly-Efficient Permutations and Its Applications (Extended Version), Cryptology ePrint Archive, Paper 2023/794 (2023).

[13] Isobe, T., Ito, R., Liu, F., Minematsu, K., Nakahashi, M., Sakamoto, K. and Shiba, R.: Areion: Highly-efficient permutations and its applications to hash functions for short input, *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, Vol. 2023, No. 2, pp. 115–154 (2023).

© 2026 Information Processing Society of Japan

付　　録

A.1　MLS 実装環境の詳細

MLS の実装には OpenMLS のバージョン 0.8.1 を使用した。暗号スイートには `MLS_128_DHKEMX25519_AES128GCM_SHA256_Ed25519` を用いた。IL2CPP 経由実行の構成では Unity Editor のバージョン 2023.1.22f1 を使用し、直接実行の構成では Android 向けにクロスコンパイルした実行ファイルを adb shell を介して起動した。

A.2　Areion 実装環境の詳細

直接実行用プログラムは Android NDK Clang 18.0.3 でビルドした。Unity アプリケーションは IL2CPP Release でビルドし、OpenXR によるステレオ描画を継続した状態で計測した。

© 2026 Information Processing Society of Japan